

PREGUNTAS & RESPUESTAS

Integrantes:

- ❖ Arteaga Sanchez Elkin
- ❖ Carbal Martínez Santiago
- ❖ Herrera Arias Heiner
- ❖ Tapias Paternina Kairo

ESCENARIO 1:

Contexto:

- PC_CLIENTE Y ATACANTE

```
Sufijo DNS específico para la conexión. . . :  
Vínculo: dirección IPv6 local. . . : fe80::35ec:8656:d711:d195%15  
Dirección IPv4. . . . . : 192.168.1.58  
Máscara de subred . . . . . : 255.255.255.0  
Puerta de enlace predeterminada . . . . . : 192.168.1.1
```

- PC_SERVIDOR

```
Adaptador de LAN inalámbrica Wi-Fi:  
  
Sufijo DNS específico para la conexión. . . :  
Vínculo: dirección IPv6 local. . . : fe80::db46:64b8:16c3:cb44%19  
Dirección IPv4. . . . . : 192.168.1.61  
Máscara de subred . . . . . : 255.255.255.0  
Puerta de enlace predeterminada . . . . . : fe80::2e96:82ff:fec0:75f8%19  
192.168.1.1
```

Proceso:

Pc servidor será el encargado de enviar los parámetros del g y p además de su llave publica

```
# Enviar los valores de g y p al cliente  
client_socket.send(f"{g},{p}".encode())  
print(f"Servidor: Parámetros g={g} y p={p} enviados al cliente")  
  
# Generar el par de claves Diffie-Hellman  
private_key, public_key = diffie_hellman_generate_keypair(p, g)  
  
# Recibir la clave pública del cliente  
public_key_client = int(client_socket.recv(1024).decode())  
print(f"Servidor: Clave pública del cliente recibida: {public_key_client}")  
  
# Enviar la clave pública al cliente  
client_socket.send(str(public_key).encode())
```

El pc cliente generara el par de llaves con los valores g y p recibidos y posteriormente enviara su llave publica al servidor

```
# Recibir los valores de g y p desde el servidor
g_p_data = client_socket.recv(1024).decode()
g, p = map(int, g_p_data.split(","))

print(f"Cliente: Parámetros recibidos g={g}, p={p}")
# Generar el par de claves Diffie-Hellman
private_key, public_key = diffie_hellman_generate_keypair(p, g)

# Enviar la clave pública al servidor
client_socket.send(str(public_key).encode())

# Recibir la clave pública del servidor
public_key_server = int(client_socket.recv(1024).decode())
print(f"Cliente: Clave pública del servidor recibida: {public_key_server}")
```

Esto se hace con el fin de que el atacante tenga la oportunidad de obtener todos los datos necesarios para el uso de sus algoritmos, debido a que, si ambos equipos tiene los parámetros en local, el ataque no sería posible.

Desarrollo:

Pc servidor inicia el programa y se conecta a pc cliente:

```
Servidor: Esperando conexión del cliente...
Servidor: Conectado a ('192.168.1.58', 50584)
Servidor: Clave pública del cliente recibida: 159
Servidor: Secreto compartido calculado: 47
Servidor: Llave simétrica derivada: 31489056e0916d59fe3add79e63f095af3ffb81604691f21cad442a85c7be617
Servidor: (1/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (2/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (3/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (4/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (5/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (6/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (7/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (8/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (9/100) Mensaje del cliente descifrado: Este es el mensaje repetido
```

Pc cliente se conecta al serve y ejecuta el envío de mensajes:

```
P5 C:\criptografia parcial\Parcial 2\escenario 1> python .\Diffie-Hellman_client.py
Cliente: Parámetros recibidos g=12, p=227
Cliente: Clave pública del servidor recibida: 176
Cliente: Secreto compartido calculado: 47
Cliente: Llave simétrica derivada: 31489056e0916d59fe3add79e63f095af3ffb81604691f21cad442a85c7be617
Cliente: (1/100) Mensaje enviado al servidor
Cliente: (2/100) Mensaje enviado al servidor
Cliente: (3/100) Mensaje enviado al servidor
Cliente: (4/100) Mensaje enviado al servidor
Cliente: (5/100) Mensaje enviado al servidor
Cliente: (6/100) Mensaje enviado al servidor
```

Tener en cuenta:

Clave publica servidor =176

Clave publica cliente = 159

Pc atacante intercepta el flujo de datos usando “Wireshark” y colocando como filtro las ips del pc servidor y pc cliente (facilita la visualización del proceso):

ip.addr==192.168.1.61 && ip.addr==192.168.1.58

Capturando desde Wi-Fi

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

ip.addr==192.168.1.61 && ip.addr==192.168.1.58

No.	Time	Source	Destination	Protocol	Length	Info
1...	64.616...	192.168.1.1...	192.168.1.1...	TCP	89	50031 → 12345 [PSH, ACK] Seq=984 Ack=3 Win=131328 Len=35
1...	64.667...	192.168.1.1...	192.168.1.1...	TCP	54	12345 → 50031 [ACK] Seq=3 Ack=1019 Win=130304 Len=0
1...	66.618...	192.168.1.1...	192.168.1.1...	TCP	89	50031 → 12345 [PSH, ACK] Seq=1019 Ack=3 Win=131328 Len=35
1...	66.668...	192.168.1.1...	192.168.1.1...	TCP	54	12345 → 50031 [ACK] Seq=3 Ack=1054 Win=130304 Len=0
1...	68.619...	192.168.1.1...	192.168.1.1...	TCP	89	50031 → 12345 [PSH, ACK] Seq=1054 Ack=3 Win=131328 Len=35
1...	68.669...	192.168.1.1...	192.168.1.1...	TCP	54	12345 → 50031 [ACK] Seq=3 Ack=1089 Win=130304 Len=0
1...	70.620...	192.168.1.1...	192.168.1.1...	TCP	89	50031 → 12345 [PSH, ACK] Seq=1089 Ack=3 Win=131328 Len=35
1...	70.685...	192.168.1.1...	192.168.1.1...	TCP	54	12345 → 50031 [ACK] Seq=3 Ack=1124 Win=130048 Len=0
1...	72.621...	192.168.1.1...	192.168.1.1...	TCP	89	50031 → 12345 [PSH, ACK] Seq=1124 Ack=3 Win=131328 Len=35
1...	72.670...	192.168.1.1...	192.168.1.1...	TCP	54	12345 → 50031 [ACK] Seq=3 Ack=1159 Win=130048 Len=0

> Frame 871: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0

> Ethernet II, Src: ChongqingFug_c5:e5:b3 (1c:bf:c0:c5:e5:b3), Dst: 192.168.1.58 (08:00:27:00:00:02)

> Internet Protocol Version 4, Src: 192.168.1.58, Dst: 192.168.1.61

> Transmission Control Protocol, Src Port: 50031, Dst Port: 12345

0000 34 68 95 76 5e 67 1c bf c0 c5 e5 b3 08 00 45 00

0010 00 34 84 e0 40 00 80 06 f2 1b c0 a8 01 3a c0 a8

0020 01 3d c3 6f 30 39 0a d5 0c 65 00 00 00 00 80 02

0030 fa f0 e5 74 00 00 02 04 05 b4 01 03 03 08 01 01

0040 04 02

El atacante sigue y observa el flujo de la información:

Wireshark · Seguir secuencia TCP (tcp.stream eq 4) · Wi-Fi

31322c323237

313539

313736

f7db416ac2a1ef010272520daec0a9c7e865fb6e4a0e94657c6ae5ac58490659162c41

5e595df8917e7ea8c70f9066bc44bec63f6e66db1dec3a78b4fea16f76e541e547596e

9f6afbfcc17795fca210e2e79ef14cb4e9c355e8aba628ef2403a997c91eec187e2727

f5281d1905133d9b3e98f976050af66569f28728cae3410a1fbb69db2abf8a3307419c

78676b1b3e7d00b3d030124c8b5b7ab097e82cdf702551272892910e013b646ab576be

b6f3b43c4c57142ddde682b629396fdb9fdb90d34506b38b9b1bd9e8d4e370d5e7aeb

b39254950bab28f0e767d77e60946c0c2baeae7dbc5114d0aa72e9c744ffeee07ac4e0

2643f0a9abfe411750d1063892171ddddd59ed7c7a5f25fa7b9137b0be6eaa12dd76f9

0b7c54039aa774562dd9fb187dba27de8b995579831ab2d20920e74374f15ad8f3e43f

f005e24fe4461569e914f1dd6495fe50369963cf4087f2930eb691ee06d91f71f61d9a

55cda72960b3934fd043f96da9db5f2b90b428de9ff702a1edcbf985e5067a5ea4fce0

4500f3554009fe76aed1d1c7c403c703134fb00eba40592b9a32921fbf7f45d418afab

1a1f68860f8763e3b44252db12095a44c99d866640b8a22cb7246561095cf13332a3b5

8d7d80e9803d3232b60a14ef9a3fab713082274c33b050a07c6a607bbce05f599f6077

52358283e043ef1beb712fb98242e06d4f01ea37c29efe7a4c6b0ca82d84030eb88998

5de27add3d0fead31e533bc74d72dc12792f5f48188554fe770f22d7fadbb99d7dc013

a6110c0f1c9505c0e66d87418a2f84dc966af1ed989931ba273e98e5a98a77f1e9e4f

8be9831b0c16889eec59cd5bb43ab1ab008c51a151c67e7e297d11ef36e2b2012280c9

90ad760ec4150713b2b22e728ab97bce6f8806de44a46518688a814e4136a1c78c4056

0369560259f1f6f6a6daf94107e5c8b75b736b3110b15880d7ccea6a03c09407ca3eda

1211e98491218b85b7ae757ba167e288ded4efedb894ea5a70d8cac62a946814565def

4eeefc7de0d34189e71ff6261892d4a72a4c73d63aa2727a6c5bdca8a0513ad8b4184b9

bb723b6e0790690cea4f9a58247a12c104a04010060df48c56b42209fb6aec3928b3a1

5fca864e86a0a4802ff61fa612d273b37346da715cad6db6f947a773159a5ec9117338

32eeaf23d5f69039a59003301c8ebc1d851ca292f61214d20f4686339d231a757c5cb6

00b4d1a03d2a0b080831a0f6690bfedc32d89f6dc6750e856f3a6350f6ef6e00132

49 client pkt(s), 2 server pkt(s), 3 turn(s).

Conversación completa (1692 bytes)

Mostrar como Raw

No delta times

Secuencia 4

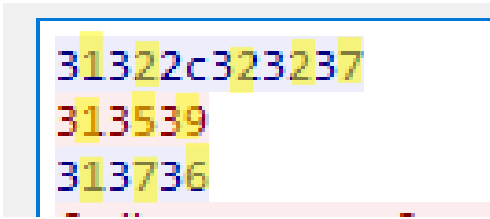
Buscar:

☐ Mayúsculas y minúsculas

Buscar siguiente

Filtrar secuencia Imprimir Guardar como... Atrás Cerrar Ayuda

El atacante identifica los datos necesarios:



Observar un patrón de 3 entre dígitos extrae los datos:

“g” = 12

“p” = 227

Clave publica servidor = 176

Clave publica cliente = 159

***Nota:** Existe el posible evento que el atacante al no conocer el código fuente que se usó pueda confundir cual es p o g si ambos son primos o incluso las llaves.*

El atacante pone en marcha su algoritmo en este caso “baby step/gigant step:

```
# Ejemplo de uso
if __name__ == "__main__":
    # Valores de ejemplo (parte del problema del logaritmo discreto)
    p = 227 # Un número primo
    g = 12  # Generador
    y = 176 # Valor público (g^x mod p)

    # Tiempo inicial
    start_time = time.time()

    # Ejecutar el algoritmo durante un máximo de 1 hora (3600 segundos)
    while True:
        x = baby_step_giant_step(g, y, p)

        if x is not None:
            print(f"El logaritmo discreto es x = {x}")
            break

    # Verificar si ha pasado 1 hora
    elapsed_time = time.time() - start_time
    if elapsed_time >= 3600:
        print("No se encontró solución en 1 hora.")
        break
```

Resultados:

```
escenario 1 > bebe-gigante.py > ...
94     else:
95         print("Atacante: No se encontró la clave privada en el tiempo dado.")
96
97     # Verificar si ha pasado 1 hora
98     elapsed_time = time.time() - start_time
99     if elapsed_time >= 3600:
100         print("No se encontró solución en 1 hora.")
101
102     # Calcular el tiempo transcurrido
103     end_time = time.time()
104     elapsed_time = end_time - start_time
105     elapsed_minutes = elapsed_time / 60 # Convertir a minutos
106
107     # Imprimir el tiempo total en minutos
108     print(f"Tiempo total de ejecución: {elapsed_minutes:.2f} minutos")
109
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

```
PS C:\criptografia_parcial\Parcial 2\escenario 1> python .\bebe-gigante.py
Atacante: Intentando encontrar la clave privada usando 'Pasos de Bebé y Pasos de Gigante'...
Atacante: ¡Clave privada encontrada! x = 76
Atacante: Secreto compartido calculado: 47
Atacante: Llave simétrica derivada: 31489056e0916d59fe3add79e63f095af3ffb81604691f21cad442a85c7be617
Atacante: Mensaje descifrado: Este es el mensaje repetido
Tiempo total de ejecución: 0.00 minutos
PS C:\criptografia_parcial\Parcial 2\escenario 1>
```

Como se observa el algoritmo tuvo rotundo éxito a la hora de encontrar x para poder hallar el secreto compartido y por consiguiente calcular la llave simétrica derivada, el atacante tomo un mensaje cifrado envidado por el flujo y este fue descifrado con éxito.

Caso: Los parámetros más grandes:

“p” =

1372645010744951812805551326739019313233321647248151333175265956275375225
6206702298960369905458848038977307901656132334347705434933645160928497114
8159280724829128531552270321268457769520042856144429883077983691811201653
4301373769199600689699905074214379584625478914259430253058101600653241459
21753228735283903

“g” =

4074656229476496537340778423455407306267407356534130335301675860934479921
0654104763969824808430330931109448281620048720300276969942539907157417365
5020138077366807935417206022265704364909016774896179119774991693342494844
7102770023916355530428049940144543734727964732283608684801296517894690465
0279473615383579

Llave publica servidor =

1324892964886468444750006877464531716267939755158160179864393847627076318
0801676265050182254745630876804475010705887861472014092390529758796250893
2485480879345895289927310572313125879217465110687773144063879345798598457
5340572199016404935498737009724949621605674940221749816020918999215379024
38439990371047633

Llave publica cliente =

1724875689247104843505151221586632073376761784313638703094534917209921306
5590867303351805375494139313923252568618672826083802633003681907100122145
2020379290045256425511875157544072876689867428558119452932275151623075795
1972465129751996242306683191473010329839239441716317245261548285215331497
0244037632252942

Pc cliente:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE

Cliente: Parámetros recibidos g=40746562294764965373407784234554073062674073565341303353016758609344799210654104763969824808430
3309311094482816200487203002769699425399071574173655020138077366807935417206022265704364909016774896179119774991693342494844710
27700239163555304280499401445437347279647322836086848012965178946904650279473615383579, p=1372645010744951812805551326739019313
2333216472481513331752659562753752256206702298960369905458848038977307901656132334347705434933645160928497114815928072482912853
1552270321268457769520042856144429883077983691811201653430137376919960068969990507421437958462547891425943025305810160065324145
921753228735283903
Cliente: Clave pública del servidor recibida: 132489296488646844475000687746453171626793975515816017986439384762707631808016762
6505018225474563087680447501070588786147201409239052975879625089324854808793458952899273105723131258792174651106877731440638793
45798598457534057219901640493549873700972494962160567494022174981602091899921537902438439990371047633
Cliente: Secreto compartido calculado: 6449600659792791568401804787063691884077445951541096706531154606527245298570867091957659
0040062679606013066853124851123788424850382911857720228653312023201368339302667998775664328324793581449301713102307744850377765
091265871718709392712584781562735412532067453842886621068899704781964500201928842465289019794
Cliente: Llave simétrica derivada: 73fce4e1540095cb45190c72f9d51cec780df9770f7352609fb07b92e57461c2
Cliente: (1/100) Mensaje enviado al servidor
Cliente: (2/100) Mensaje enviado al servidor
Cliente: (3/100) Mensaje enviado al servidor
Cliente: (4/100) Mensaje enviado al servidor
Cliente: (5/100) Mensaje enviado al servidor
Cliente: (6/100) Mensaje enviado al servidor
Cliente: (7/100) Mensaje enviado al servidor
Cliente: (8/100) Mensaje enviado al servidor
Cliente: (9/100) Mensaje enviado al servidor
Cliente: (10/100) Mensaje enviado al servidor
Cliente: (11/100) Mensaje enviado al servidor
Cliente: (12/100) Mensaje enviado al servidor
Cliente: (13/100) Mensaje enviado al servidor
```

Pc server:

```
PS C:\cosa de santiago(borrar)\parcial 2\escenario 1> python .\Diffie-Hellman_server.py
Servidor: Esperando conexión del cliente...
Servidor: Conectado a ('192.168.1.58', 50960)
Servidor: Clave pública del cliente recibida: 1724875689247104843505151221586632073376761784313638703094534917209921306
59086730335180537549413931392325256861867282608380263300368190710012214520203792900452564255118751575440728766898674285
811945293227515162307579519724651297519962423066831914730103298392394417163172452615482852153314970244037632252942
Servidor: Secreto compartido calculado: 6449600659792791568401804787063691884077445951541096706531154606527245298570867
91957659004006267960601306685312485112378842485038291185772022865331202320136833930266799877566432832479358144930171316
807744850377765091265871718709392712584781562735412532067453842886621068899704781964500201928842465289019794
Servidor: Llave simétrica derivada: 73Fce4e1540095cb45190c72f9d51cec780df9770f7352609fb07b92e57461c2
Servidor: (1/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (2/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (3/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (4/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (5/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (6/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (7/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (8/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (9/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (10/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (11/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (12/100) Mensaje del cliente descifrado: Este es el mensaje repetido
Servidor: (13/100) Mensaje del cliente descifrado: Este es el mensaje repetido
```

Pc atacante:

Wireshark · Seguir secuencia TCP (tcp.stream eq 7) · Wi-Fi

14 client pkt(s), 2 server pkt(s), 3 turn(s).

Conversación completa (1690 bytes) Mostrar como Raw No delta times Secuencia 7

Buscar: ☐ Mayúsculas y minúsculas

```
343037343635363232393437363439363533373334303737383432333435353430373330363236373430373335363533343133303333353330313637353
836303933343437393932313036353431303437363339363938323438303834333033333039333131303934343832383136323030343837323033303032
373639363939343235333939303731353734313733363535303230313338303737333636383037393335343137323036303232323635373034333634393
039303136373734383936313739313139373734393931363933333432343934383434373130323737303032333931363335353533303432383034393934
303134343534333733343732373936343733323238333630383638343830313239363531373839343639303436353032373934373336313533383335373
92c133373236343530313037343439353138313238303535353133323637333930313933313332333332313634373234383135313333333137353236
353935363237353337353232353632303637303232393839363033363939303534353838343830333839373733303739303136353631333233333433343
737303534333439333336343531363039323834393731313438313539323830373234383239313238353331353532323730333231323638343537373639
353230303432383536313434343239383833303737393833363931383131323031363533343330313337333736393139393630303638393639393930353
037343231343337393538343632353437383931343235393433303235333035383130313630303635333234313435393231373533323238373335323833
393033
313732343837353638393234373130343834333530353135313232313538363633323037333337363736313738343331333633383730333039343533343
931373230393932313330363535393038363733303333353138303533373534393431333933313339323332353235363836313836373238323630383338
303236333330303336383139303731303031323231343532303230333739323930303435323536343235353131383735313537353434303732383736363
839383637343238353538313139343532393332323735313531363233303735373935313937323436353132393735313939363234323330363638333139
313437333031303332393833393233393434313731363331373234353236313534383238353231353333313439373032343430333736333232353239343
2
313332343839323936343838363436383434343735303030363837373436343533313731363236373933393735353135383136303137393836343339333
834373632373037363331383038303136373632363530353031383232353437343536333038373638303434373530313037303538383738363134373230
313430393233393035323937353837393632353038393332343835343830383739333435383935323839393237333130353732333133313235383739323
137343635313130363837373733313434303633383739333435373938353938343537353334303537323139393031363430343933353439383733373030
393732343934393632313630353637343934303232313734393831363032303931383939393231353337393032343338343339393930333731303437363
333
84a0e45f82897acd3fcd7c5e2fbd967286ad470768b856b2afaa2b100f43172e0b4e55
88dc26a35bb79aa1227c1df6b80031e95aaea4b8a8f8143ea85a702f46bd5ca6df3d22
faf3844a6e4e2a4310de62b9b33cfd986a5d4c45ed8e29ecafa7812ec7dbbfac9d1ab
a160f239cab580fea664d58cc95d420d3d58951557cb39a4997e19378cef0b0378ef67
dc6f28f08b9e2d4ab32146174f26a8acd59a40aeb18c284a4da5cc4157c9cae084e64d
025f2a24657f61a012377708010b1b32360a350405c7a05f43538e4d04fc88087
```

El atacante realiza el mismo procedimiento para intentar hallar m

```
ial/Parcial 2/escenario 1/bebe-gigante.py"
Atacante: Intentando encontrar la clave privada usando 'Pasos de Bebé y Pasos de Gigante'...
[]
```


Resultados:



```
PS C:\criptografia_parcial\Parcial 2\escenario 1> python .\bebe-gigante.py
Atacante: Intentando encontrar la clave privada usando 'Pasos de Bebé y Pasos de Gigante'...
Traceback (most recent call last):
PS C:\criptografia_parcial\Parcial 2\escenario 1>
PS C:\criptografia_parcial\Parcial 2\escenario 1> python .\bebe-gigante.py
Atacante: Intentando encontrar la clave privada usando 'Pasos de Bebé y Pasos de Gigante'...
Traceback (most recent call last):
PS C:\criptografia_parcial\Parcial 2\escenario 1> python .\bebe-gigante.py
Atacante: Intentando encontrar la clave privada usando 'Pasos de Bebé y Pasos de Gigante'...
Traceback (most recent call last):
PS C:\criptografia_parcial\Parcial 2\escenario 1> python .\bebe-gigante.py
```

No hubo suerte en el lapso de 1 hora

CONCLUSIONES.

¿Qué factores influyen en la dificultad de resolver el problema del logaritmo discreto utilizando el algoritmo de "baby-step giant-step"?

La dificultad de resolver el problema del logaritmo discreto depende principalmente del tamaño de p y g . Para valores pequeños, como en este caso ($p = 227$), el ataque es más factible, ya que el número de posibles soluciones es reducido. A medida que p y g crecen, el espacio de búsqueda para el algoritmo "baby-step giant-step" se incrementa exponencialmente, lo que hace más difícil y lento el cálculo. Además, el rendimiento del ataque también depende de la eficiencia del cálculo modular y la implementación del algoritmo. La elección de g puede afectar la rapidez del ataque si se elige de forma inadecuada.

¿Cuáles son los beneficios y desventajas de utilizar Diffie-Hellman sobre un grupo cíclico $\mathbb{F}_p$ en comparación con otros métodos de intercambio de claves?*

Beneficios: Simplicidad: Diffie-Hellman sobre grupos cíclicos es simple de implementar y es bien entendido en la criptografía. Seguridad probada: Para grupos suficientemente grandes, la seguridad es robusta y resistirá la mayoría de los ataques conocidos. Compatibilidad: Es ampliamente soportado en muchas aplicaciones y protocolos (e.g., TLS).

Desventajas: Vulnerabilidad al logaritmo discreto: Como vimos en este escenario, grupos pequeños permiten ataques efectivos como "baby-step giant-step". Para mitigar esto, los grupos deben ser suficientemente grandes, lo que puede hacer las operaciones más lentas. Curvas elípticas más eficientes: Los protocolos de intercambio de claves basados en curvas elípticas (EC-DH) proporcionan el mismo nivel de seguridad con menores tamaños de clave, lo que los hace más eficientes en términos de tiempo y ancho de banda.

Escenario 2

Contexto:

- PC_CLIENTE Y ATACANTE

```
Sufijo DNS específico para la conexión. . . :  
Vínculo: dirección IPv6 local. . . : fe80::35ec:8656:d711:d195%15  
Dirección IPv4. . . . . : 192.168.1.58  
Máscara de subred . . . . . : 255.255.255.0  
Puerta de enlace predeterminada . . . . . : 192.168.1.1
```

- PC_SERVIDOR

Adaptador de LAN inalámbrica Wi-Fi:

```
Sufijo DNS específico para la conexión. . . :  
Vínculo: dirección IPv6 local. . . : fe80::db46:64b8:16c3:cb44%19  
Dirección IPv4. . . . . : 192.168.1.61  
Máscara de subred . . . . . : 255.255.255.0  
Puerta de enlace predeterminada . . . . . : fe80::2e96:82ff:fec0:75f8%19  
192.168.1.1
```

Desarrollo:

En este escenario haremos una simulación de un “MitM” primero se explicara cómo funciona el escenario planteado.



Esto provoca que el atacante tenga total control del flujo de información, puedo tanto ver mensaje como alterarlos para esto el cliente se debería de conectar al atacante en lugar al servidor.

SERVIDOR

```
def server():
    # Crear el servidor
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind(('0.0.0.0', 12345))
    server_socket.listen(1)
    print("Servidor: Esperando conexión del cliente...")

    client_socket, addr = server_socket.accept()
    print(f"Servidor: Conectado a {addr}")

    # Generar el par de claves Diffie-Hellman (curva P-256)
    private_key, public_key = generate_keypair()

    # Enviar la clave pública al cliente
    public_bytes = public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
```

CLIENTE

```
def client():
    # Crear el cliente y conectarse al servidor
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client_socket.connect(('localhost', 12345))

    # Generar el par de claves Diffie-Hellman (curva P-256)
    private_key, public_key = generate_keypair()

    # Recibir la clave pública del servidor
    server_public_bytes = client_socket.recv(1024)
    server_public_key = serialization.load_pem_public_key(server_public_bytes, backend=default_backend())
    print("Cliente: Clave pública del servidor recibida.")

    # Enviar la clave pública al servidor
    public_bytes = public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    )
    client_socket.send(public_bytes)
    print("Cliente: Clave pública enviada al servidor.")
```

ATACANTE

```
def mitm():
    # Crear sockets para el cliente y el servidor
    mitm_server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    mitm_client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    # Conectar al servidor
    mitm_server_socket.connect(('192.168.1.61', 12345))
    print("Atacante: Conectado al servidor.")

    # Configurar escucha para que el cliente se conecte al atacante
    mitm_client_socket.bind(('0.0.0.0', 12345)) # Puerto diferente para que el cliente se conecte al atacante
    mitm_client_socket.listen(1)
    print("Atacante: Esperando conexión del cliente...")

    # Aceptar la conexión del cliente
    client_socket, addr = mitm_client_socket.accept()
    print(f"Atacante: Conectado al cliente en {addr}.")

    # Generar el par de claves de Eve
    private_key_eve, public_key_eve = generate_keypair()
```

Luego se aplica el protocolo de intercambio de llaves Diffie-Hellman utilizando la curva elíptica P256, pero esto no es suficiente debido a que el atacante tiene tanto las llaves públicas como el KDF debido a que se encuentra en medio de la conexión. En adición tanto el cliente como servidor no tienen idea o información de que el atacante existe.

```

# Generar el par de claves de Eve
private_key_eve, public_key_eve = generate_keypair()

# Interceptar y enviar claves
# Recibir la clave pública del servidor
server_public_bytes = mitm_server_socket.recv(1024)
server_public_key = serialization.load_pem_public_key(server_public_bytes, backend=default_backend())

# Enviar la clave pública de Eve al cliente
client_socket.send(public_key_eve.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
))

# Recibir la clave pública del cliente
client_public_bytes = client_socket.recv(1024)
client_public_key = serialization.load_pem_public_key(client_public_bytes, backend=default_backend())

# Enviar la clave pública del servidor a Eve
mitm_server_socket.send(public_key_eve.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
))

```

Posteriormente el atacante genera un par de llaves y lo que surge ahora es que el atacante conoce la llave pública del cliente por lo que con tan solo pasárselo al servidor el servidor le devolver su llave pública por lo que ahora el atacante tendrá acceso a ambas claves públicas como el KDF y con su propia llave pública mantiene la conexión con el cliente, por decirlo así está modificando el flujo de datos a su conveniencia.

Resultados

CLIENTE

```
PS C:\criptografia_parcial\Parcial 2\escenario 2> python .\ClienteP256.py
Cliente: Clave pública del servidor recibida.
Cliente: Clave pública enviada al servidor.
Cliente: Llave simétrica derivada: ee77d6dd577724b496f194c40c2244c83260117451981217528e81f2bad5c583
Cliente: Respuesta del servidor descifrada (1): Mensaje 1 recibido correctamente.
Cliente: Respuesta del servidor descifrada (2): Mensaje 2 recibido correctamente.
Cliente: Respuesta del servidor descifrada (3): Mensaje 3 recibido correctamente.
Cliente: Respuesta del servidor descifrada (4): Mensaje 4 recibido correctamente.
Cliente: Respuesta del servidor descifrada (5): Mensaje 5 recibido correctamente.
Cliente: Respuesta del servidor descifrada (6): Mensaje 6 recibido correctamente.
Cliente: Respuesta del servidor descifrada (7): Mensaje 7 recibido correctamente.
Cliente: Respuesta del servidor descifrada (8): Mensaje 8 recibido correctamente.
Cliente: Respuesta del servidor descifrada (9): Mensaje 9 recibido correctamente.
Cliente: Respuesta del servidor descifrada (10): Mensaje 10 recibido correctamente.
```

SERVIDOR

```
PS C:\cosa de santiago(borrar)\parcial 2\escenario 2> python .\ServidorP256.py
Servidor: Esperando conexión del cliente...
Servidor: Conectado a ('192.168.1.58', 61843)
Servidor: Clave pública enviada al cliente.
Servidor: Clave pública del cliente recibida.
Servidor: Llave simétrica derivada: 23fdaf3bf8c3234c9753ee8ee2a0ef266b102448067df538104445eb37e6ee90
Servidor: Mensaje descifrado (1): Mensaje secreto 1
Servidor: Mensaje descifrado (2): Mensaje secreto 2
Servidor: Mensaje descifrado (3): Mensaje secreto 3
Servidor: Mensaje descifrado (4): Mensaje secreto 4
Servidor: Mensaje descifrado (5): Mensaje secreto 5
Servidor: Mensaje descifrado (6): Mensaje secreto 6
Servidor: Mensaje descifrado (7): Mensaje secreto 7
Servidor: Mensaje descifrado (8): Mensaje secreto 8
Servidor: Mensaje descifrado (9): Mensaje secreto 9
Servidor: Mensaje descifrado (10): Mensaje secreto 10
Servidor: Mensaje descifrado (11): Mensaje secreto 11
Servidor: Mensaje descifrado (12): Mensaje secreto 12
Servidor: Mensaje descifrado (13): Mensaje secreto 13
Servidor: Mensaje descifrado (14): Mensaje secreto 14
Servidor: Mensaje descifrado (15): Mensaje secreto 15
Servidor: Mensaje descifrado (16): Mensaje secreto 16
```

ATACANTE

```
PS C:\criptografia_parcial\Parcial 2\escenario 2> python .\AtacanteP256.py
Atacante: Conectado al servidor.
Atacante: Esperando conexión del cliente...
Atacante: Conectado al cliente en ('127.0.0.1', 61842).
Atacante: Llave derivada con servidor 23fdaf3bf8c3234c9753ee8ee2a0ef266b102448067df538104445eb37e6ee90
Atacante: Llave derivada con cliente ee77d6dd577724b496f194c40c2244c83260117451981217528e81f2bad5c583
Atacante: Mensaje del cliente interceptado: Mensaje secreto 1
Atacante: Respuesta del servidor interceptada: Mensaje 1 recibido correctamente.
Atacante: Mensaje del cliente interceptado: Mensaje secreto 2
Atacante: Respuesta del servidor interceptada: Mensaje 2 recibido correctamente.
Atacante: Mensaje del cliente interceptado: Mensaje secreto 3
Atacante: Respuesta del servidor interceptada: Mensaje 3 recibido correctamente.
Atacante: Mensaje del cliente interceptado: Mensaje secreto 4
Atacante: Respuesta del servidor interceptada: Mensaje 4 recibido correctamente.
Atacante: Mensaje del cliente interceptado: Mensaje secreto 5
Atacante: Respuesta del servidor interceptada: Mensaje 5 recibido correctamente.
```

FLUJO DE DATOS



CONCLUSIONES

¿Qué vulnerabilidades inherentes a Diffie-Hellman sobre curvas elípticas se pueden explotar en un ataque de hombre en el medio?

La principal vulnerabilidad es que el protocolo de intercambio de llaves Diffie-Hellman no autentica las claves públicas. Esto significa que un atacante puede interceptar y modificar el intercambio de claves sin ser detectado. Al suplantar las claves públicas, el atacante puede establecer conexiones separadas con ambas partes, obteniendo claves compartidas distintas, lo que le permite descifrar y modificar la comunicación.

¿Qué contramedidas podrían implementarse para mitigar estos ataques en un entorno real?

Autenticación de Claves: Implementar un mecanismo de autenticación (por ejemplo, firmas digitales) para asegurar que las claves públicas provengan de fuentes confiables. Protocolos de intercambio de llaves autenticados: Utilizar protocolos como TLS, que incorporan autenticación y cifrado, protegiendo así contra ataques MitM. Verificación en dos fases: Requerir que los participantes validen las claves públicas a través de un canal seguro o utilizando un método de verificación alternativo, como un código QR o una llamada telefónica.

ESCENARIO 3

SERVIDOR Y CLIENTE

```
Sufijo DNS específico para la conexión. . :  
Vínculo: dirección IPv6 local. . . : fe80::35ec:8656:d711:d195%15  
Dirección IPv4. . . . . : 192.168.1.58  
Máscara de subred . . . . . : 255.255.255.0  
Puerta de enlace predeterminada . . . . . : 192.168.1.1
```

Resultados Diffie-Hellman sobre el grupo cíclico:

```
PS C:\criptografia_parcial\Parcial 2\escenario 1> python .\Diffie-Hellman_server.py  
Servidor: Esperando conexión del cliente...  
Servidor: Conectado a ('127.0.0.1', 62239)  
Servidor: Parámetros g=12 y p=227 enviados al cliente  
Servidor: Clave pública del cliente recibida: 104  
Servidor: Secreto compartido calculado: 30  
Servidor: Llave simétrica derivada: 624b60c58c9d8bfb6ff1886c2fd605d2adeb6ea4da576068201b6c6958ce93f4  
Servidor: (1/100) Mensaje del cliente descifrado: Este es el mensaje repetido  
Total de bytes enviados: 8  
Total de bytes recibidos: 38
```

Resultados Diffie-Hellman utilizando la curva elíptica P256:

```
PS C:\criptografia_parcial\Parcial 2\escenario 2> python .\ServidorP256.py  
Servidor: Esperando conexión del cliente...  
Servidor: Conectado a ('127.0.0.1', 62260)  
Servidor: Clave pública enviada al cliente.  
Servidor: Clave pública del cliente recibida.  
Servidor: Llave simétrica derivada: 42d5028304a896888793a863f7cb54f1a72d805fb268704f237b5e8447ea5ce0  
Servidor: Mensaje descifrado (1): Mensaje secreto 1  
Total de bytes enviados: 242  
Total de bytes recibidos: 226  
PS C:\criptografia_parcial\Parcial 2\escenario 2> █
```

Resultados RSA:

```
PS C:\criptografia_parcial\Parcial 2\escenario 3> python .\ServidorRSA.py  
Servidor RSA: Esperando conexión...  
Servidor RSA: Conectado con ('127.0.0.1', 62192)  
Servidor RSA: Clave pública enviada al cliente (451 bytes).  
Servidor RSA: Mensaje cifrado recibido (256 bytes).  
Servidor RSA: Mensaje descifrado (1/50): Este es el mensaje número 1 secreto usando RSA OAEP.  
Servidor RSA: Mensaje cifrado recibido (768 bytes).
```

Resultados Gamal:

```
PS C:\criptografia_parcial\Parcial 2\escenario 3> python .\ServidorGamal.py  
Servidor ElGamal en espera...  
Conexión establecida con: ('127.0.0.1', 63314)  
Cliente: Este es un mensaje secreto usando ElGamal.  
Total de información recibida: 416 bytes  
Total de información enviada: 56 bytes
```


¿Cuáles son las principales diferencias en términos de eficiencia y seguridad entre RSA OAEP y ElGamal? Justifica cuál criptosistema asimétrico sería más adecuado en un contexto donde el tamaño de los mensajes y la rapidez de la comunicación son críticos.

Diferencias en eficiencia: RSA OAEP es más lento en términos de cifrado y descifrado. Sin embargo, proporciona una buena seguridad cuando se utiliza con claves largas. ElGamal, por otro lado, es más eficiente y rápido, especialmente para el cifrado, lo que lo hace más adecuado para aplicaciones donde la velocidad es crucial. Diferencias en seguridad: RSA depende de la dificultad de factorizar números grandes, mientras que ElGamal se basa en la dificultad del problema del logaritmo discreto. Conclusión: En situaciones donde el tamaño de los mensajes y la rapidez son críticos, ElGamal sería más adecuado debido a su eficiencia en el cifrado y descifrado.

En base a la comparación con las comunicaciones simétricas de los escenarios anteriores, ¿qué conclusiones puedes extraer sobre el uso de cifrado simétrico vs. asimétrico en aplicaciones de comunicación de red?

Cifrado simétrico (Salsa20 y AES-256) es generalmente más rápido y eficiente en términos de tiempo y tamaño de los mensajes. Es ideal para cifrar grandes volúmenes de datos. Cifrado asimétrico (RSA y ElGamal), aunque más seguro para el intercambio de claves, es significativamente más lento. Por lo tanto, en la práctica, el cifrado asimétrico a menudo se usa para intercambiar claves simétricas, y luego se utiliza cifrado simétrico para la transmisión de datos reales.

CONCLUSIONES

Resumen de los principales aprendizajes en relación con la seguridad y la eficiencia de los distintos esquemas criptográficos implementados: Diffie-Hellman (DH): Este método permite el intercambio de claves de forma segura, pero su vulnerabilidad al logaritmo discreto puede ser explotada si los parámetros no son suficientemente grandes. Es más eficiente en términos de velocidad, pero menos seguro sin autenticación. Curvas Elípticas: La implementación de DH sobre curvas elípticas (ECDH) proporciona la misma seguridad que DH convencional con tamaños de clave más pequeños, lo que mejora la eficiencia. Sin embargo, también es vulnerable a ataques como el hombre en el medio si no se implementa correctamente. RSA OAEP: Este esquema es robusto en términos de seguridad, pero más lento en comparación con los métodos de cifrado simétrico y ElGamal. El tamaño de la clave juega un papel crucial en la eficiencia y la seguridad. ElGamal: Proporciona una buena combinación de seguridad y eficiencia, siendo más rápido en el cifrado que RSA, pero su tamaño de mensaje puede ser mayor. Es particularmente útil en situaciones donde se requiere velocidad. Cifrado Simétrico (Salsa20 y AES-256): Estos métodos son mucho más rápidos y eficientes para cifrar grandes volúmenes de datos, lo que los hace ideales para aplicaciones de comunicación de red donde la rapidez es esencial. Relevancia de la correcta implementación de protocolos criptográficos y de los posibles ataques: La implementación correcta de protocolos criptográficos es crucial para asegurar la privacidad y confidencialidad de la información. Sin autenticación de las claves públicas, como se demostró en el ataque MitM, un atacante puede interceptar y modificar la comunicación sin que las partes se den cuenta. Los ataques, como el hombre en el medio, subrayan la importancia de la autenticación y la verificación de identidades en cualquier sistema de comunicación. La falta de medidas adecuadas puede llevar a compromisos de seguridad graves, exponiendo información sensible y sistemas críticos. Recomendaciones para garantizar un intercambio seguro de llaves y una comunicación cifrada robusta: Autenticación de Claves: Implementar un mecanismo de autenticación, como firmas digitales o certificados, para verificar la autenticidad de las claves públicas antes de usarlas en el intercambio. Uso de Protocolos Establecidos: Adopta protocolos bien establecidos como TLS, que incluyen autenticación y cifrado, para proteger las comunicaciones en red. Cifrado Híbrido: Utiliza un enfoque híbrido donde el cifrado asimétrico se utiliza para intercambiar claves simétricas, y luego se utiliza cifrado simétrico para la comunicación real, aprovechando la eficiencia de ambos métodos. Revisión y Auditoría de Seguridad: Realizar revisiones periódicas y auditorías de seguridad en los sistemas para identificar vulnerabilidades y garantizar que se implementen las mejores prácticas de seguridad. Educación y Conciencia: Educar a los usuarios sobre las mejores prácticas en seguridad y la importancia de utilizar medidas de protección, como contraseñas fuertes y autenticación de dos factores, para minimizar el riesgo de ataques.

Enlace del repositorio: <https://github.com/scarbal/salsa20-AES-256.git>